

1. Import Libraries and Load Image

```
from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()

filename = next(iter(uploaded))

image = cv2.imread(filename)

if image is None:
    print("Error: Could not read the image.")
else:
    print("Image loaded successfully!")
    print("Filename:", filename)
    print("Image shape:", image.shape)

    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis("off")
    plt.show()
```

[Choose Files](#) Screenshot... 015932.png

Screenshot 2026-04-30 015932.png(image/png) - 804591 bytes, last modified: 4/30/2026 - 100% done

Saving Screenshot 2026-04-30 015932.png to Screenshot 2026-04-30 015932.png

Image loaded successfully!

Filename: Screenshot 2026-04-30 015932.png

Image shape: (1036, 1883, 3)

Original Image



2. Examine Image Properties

```
height, width, channels = image.shape

print("Width:", width)
print("Height:", height)
print("Number of Channels:", channels)
```

```
print("Data Type:", image.dtype)
print("Resolution:", width, "x", height)
```

```
Width: 1883
Height: 1036
Number of Channels: 3
Data Type: uint8
Resolution: 1883 x 1036
```

3. Save Image in JPEG and PNG

```
cv2.imwrite("output_image.jpg", image)
cv2.imwrite("output_image.png", image)

print("Image saved successfully in JPEG and PNG formats.")
```

```
Image saved successfully in JPEG and PNG formats.
```

4. Convert Image to Different Color Spaces

```
# Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# LAB
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
```

Display

```
plt.figure(figsize=(12, 8))

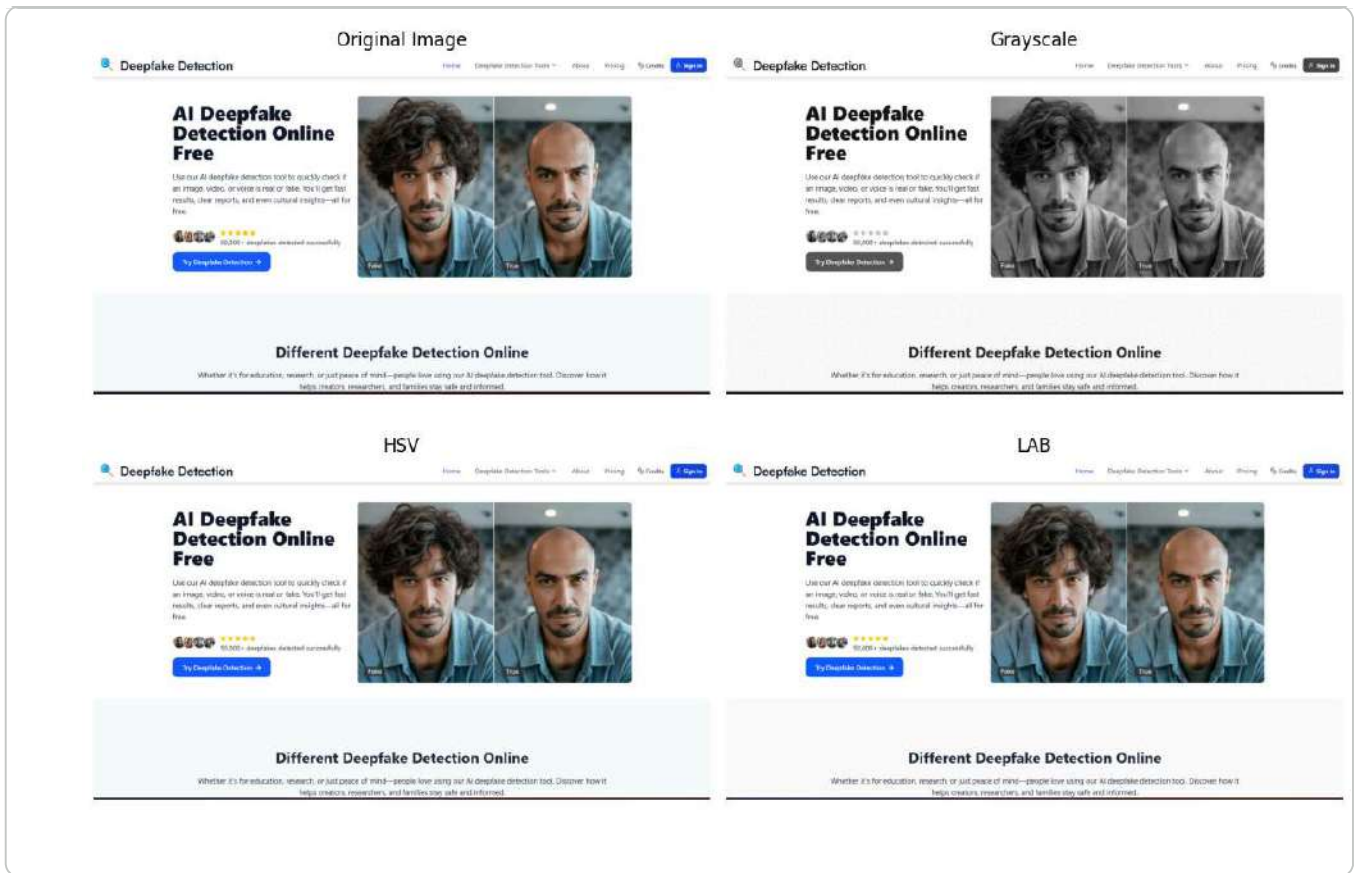
plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()
```



5. Resize, Rotate and Flip

```
resized = cv2.resize(image, (500, 400))
```

Rotate

```
(h, w) = image.shape[:2]
center = (w // 2, h // 2)

matrix = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(image, matrix, (w, h))
```

Horizontal Flip

```
horizontal_flip = cv2.flip(image, 1)
```

Vertical Flip

```
vertical_flip = cv2.flip(image, 0)
```

Display

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
```

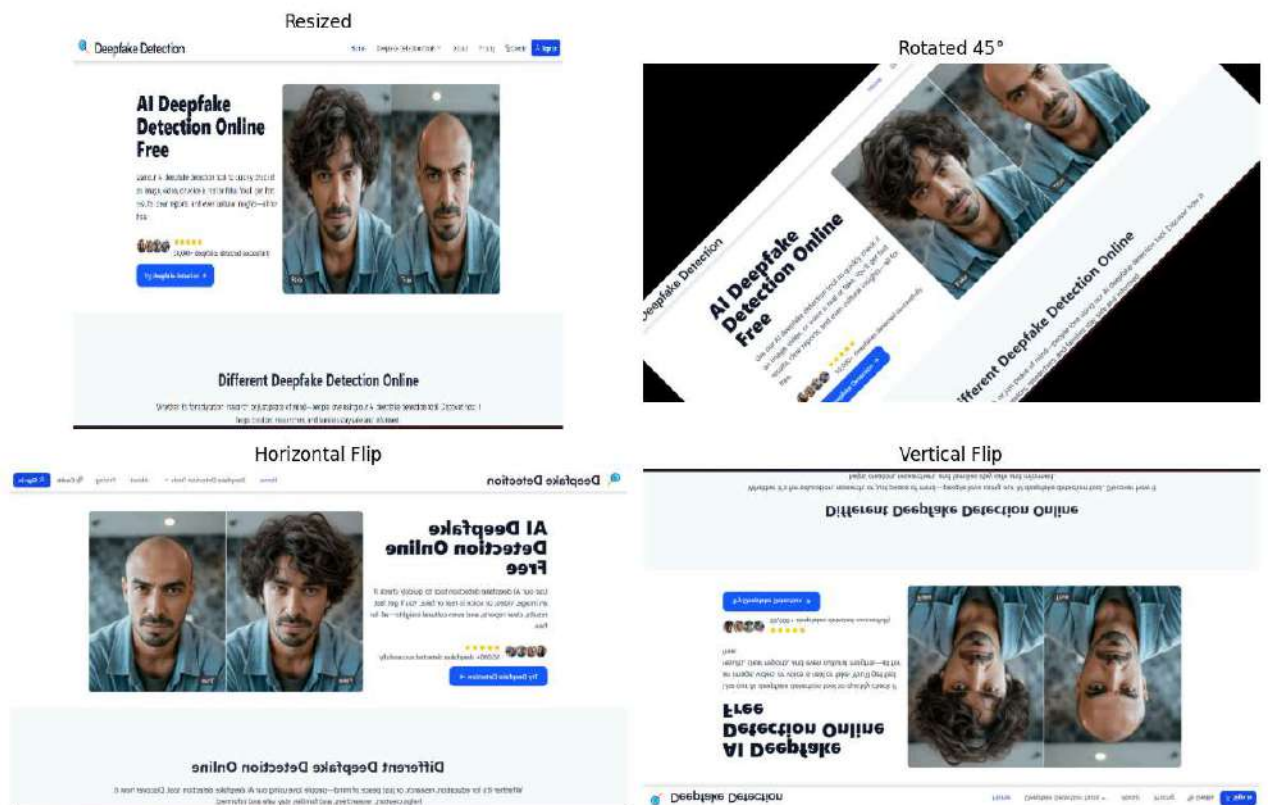
```
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated 45°")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()
```



6. Generate Image Complement / Negative

```
negative = 255 - image

plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")
plt.show()
```

Negative Image



7. Crop a Region of Interest (ROI)

```
# Crop region
roi = image[100:400, 100:500]

plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

print("ROI Shape:", roi.shape)
```

Region of Interest (ROI)

**AI Deepf
Detection
Free**

Use our AI deepfake detec

ROI Shape: (300, 400, 3)

8. Display Original and Processed Images Together

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
```

```
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("ROI")
plt.axis("off")

plt.tight_layout()
plt.show()
```

